Project Name: Seeking Tutor Problem

Submitted to

Dr. Md. Nawab Yousuf Ali

Professor

Submitted By

1

Ratry

ID : 2020-3-60-002

2

Rifa Azad

ID: 2022-1-60-265

3

Abir Hasan

ID: 2021-1-60-035

Navigating the Intricacies of Tutor-Student Synchronization

A Three-Step Approach

Synchronization for the Seeking-Tutor-Problem

01 Resource Allocation

The core problem is to ensure that the tutor and student are coordinated, considering a limited number of waiting room chairs by Coordinator

02 Interaction & Synchronization

Simulate the flow of students in the waiting room and ensure proper synchronization between the tutor and student, so that no teacher remains idle when students are waiting in the waiting room.

03 Termination

When a student is done seeking service, will terminate.
When the tutor gives service to all the students, he will terminate himself
Coordinate will terminate after assigning all students to tutors and when the waiting room is empty

Workflow

Input Initialization
01

Gather input for the total number of student and chairs

Semaphores
02

Initialize semaphores for tutor, students and coordinator

Mutex Locks
03

Set up mutex locks for student management and waiting room management

Threads
04

Create threads for the tutor, students and coordinator

Interaction & Synchronization
05

Handle the tutors from idling when students are waiting for service & notified by coordinator by an arriving student.

Flow & Waiting Room Optimization
06

Assess chair availability to direct incoming student to either wait or leave.

Thread Termination
07

"Terminate student threads and end the tutor thread after all students are done along with coordinator

Resource Deallocation
08

End the main program to free up all allocated resources

```
Pseudocode for Tutoring Simulation Program

1. Define Constants:
   - MAX_STUDENT
   - MAX_TUTOR
   - MAX_CHAIR

2. Declare Global Variables:
   - total_chair, total_student, total_tutor, help_limit
   - req_order, helped_student, occupied_chair
   - Arrays: priority, visited_q, total_visit, tutor_ID, student_ID, time_arrive
   - Semaphores: student_s, coordinator_s, tutored_student[], mutex_lock
```

```
3. Define Thread Functions:
   a. student_t (student_id):
      - Repeat:
        - If student reached help_limit:
          - Increment helped_student count
          - If all students helped, display message
          - Exit thread
        - Wait if no chair available
        - Occupy chair and update request order
        - Wait for tutoring
        - Increment total visits for student

   b. tutor_t (tutor_id):
      - Repeat:
        - Exit if all students are helped
        - Find the student with highest priority and least arrival time from the waiting
room    - Tutor the student
        - Release the student's semaphore

   c. coordinator_t:
      - Repeat:
        - Exit if all students are helped
        - Wait for a student to sit in a chair
        - Assign priority and time of arrival for the waiting student
        - Signal tutor semaphore
```

4. Function start_simulation:

- Input total number of students, tutors, chairs, and help limit from user inputInitialize semaphores
- Initialize arrays for tracking students and visits
- Create threads for students, tutors, and coordinator
- Join all threads

5. Main Function:

- Display a menu to start the simulation or exit the program
- Based on user choice, either start the simulation or exit

OUTPUT ANALYSIS

SEQUENTIAL FLOW

“Student arrive, takes a sit, coordinator assign to tutor, tutor helps, help completed and student terminated and continue till all student are done”

CONCURRENT ACTIVITIES

“Multiple student waiting in the waiting room, coordinate assign student to tutor, tutor is busy, providing help to student”

LIMIT CASES

“When the waiting room is full, student can not entire the waiting room, also when all tutor are booked, no student can be assign to tutor by coordinate”

SERVICE COUNT

“The number of Service provided is determined by `helped_ student` and number of student leaving the room, keeping the count of `tutored_ student`”

STUDENT TURNOVER

“The number of students who arrived should be equal to helped student plus those who left due to lack of chairs”

PROGRAM TERMINATION

“Program should terminate gracefully after all the students has been provided service, coordinated has assign all the student to the tutors and tutors has completed all his services to all his students. After no student available in the waiting room for tutorial.”

Enhancements

Adaptability

Enable dynamic adjustments to program parameters during execution for increased flexibility. This will make it closer to a real-world scenario and also make the scheduling part more challenging

Increase resources

Another coordinate or new more tutors can decrease the waiting time for students making less traffic in waiting room

Error Resilience

Improve error handling mechanisms to enhance the overall robustness and reliability of the program.